Project Phase 4

Database Implementation and Web Application

Problem Statement

Submission Deadline: 12th November 2026, 11:00 PM

---

1 Introduction and Objectives

This final project phase marks the culmination of your design efforts. In this phase you take your own Mini World back, review what two different teams did with it, and turn its Phase 3 design into a functional, populated database system with applications.

You may implement your database using **any of the database paradigms covered in the tutorials**: relational (MySQL), document (MongoDB) or graph (Neo4j). You will populate it with comprehensive data taken from the website itself, using GenAI tools if needed, and build a **web application**, modelled on the real-world website of your Mini World, through which users interact with this data. You should replicate the results given by the website, where feasible.

This phase tests your proficiency in database creation, data management, and the practical use of a native query language (SQL, the MongoDB Query Language or Cypher) for data retrieval and manipulation.

Example code for every part of this phase is provided in the companion document *Project Phase 4: Boilerplate Code*.

Core Objectives

1. **Database Implementation:** Correctly translate the Phase 3 schema of your Mini World into your chosen data model: `CREATE TABLE` statements in MySQL; collections with validators and indexes in MongoDB; or node labels, relationship types, constraints and indexes in Neo4j. Keys, references, constraints and data types must be enforced accurately.
2. **Data Population:** Populate your database with a substantial and coherent set of data that reflects your Mini World. Extract data from the website, using Generative AI tools if needed. The data must be sufficient to test all functional requirements meaningfully.
3. **Application Development:** Build a web application with a Python backend, modelled on the original website your Mini World is based on.
4. **Functional Requirements:** Implement all functional requirements of your project, including a minimum of **5 distinct queries** and **3 update operations**. Each must use raw queries in your database's native query language, run from your Python backend and triggered through the web interface.

Working From Another Team's Design

- **Baseline:** use the Phase 3 schema as your baseline. Try to keep it unchanged as much as possible.
- **Revisions:** you may revise it, for example to fix errors, or to restore requirements from your Phase 1 version that were lost or misread.

- **Documentation:** list every significant change, with its reason, in the *Design Changes* section of your `README.md` (see Section 5).

2 Choosing a Database Paradigm

Choose **any one** of the following paradigms, using the tools and versions shown in the tutorials:

Paradigm	DBMS	Query language	Python driver	Script files
Relational	MySQL	SQL	`PyMySQL` or `mysql-connector-python`	`schema.sql` `populate.sql`
Document	MongoDB	MQL	`pymongo`	`schema.js` `populate.js`
Graph	Neo4j	Cypher	`neo4j` (official driver)	`schema.cypher` `populate.cypher`

MQL: the MongoDB Query Language (filter documents, update documents and aggregation pipelines).

Mapping from Phase 3. The Phase 3 schema of your Mini World is relational. If you choose MongoDB or Neo4j, your `README.md` must include a short *Schema Mapping* section that explains:

- how each Phase 3 relation maps to your model (embedded vs. referenced documents in MongoDB; nodes vs. relationships vs. properties in Neo4j);
- how each key and integrity constraint from Phase 3 is enforced. If the DBMS cannot enforce a constraint natively, mention where/how your application enforces it instead.

3 Part A: Database Implementation

This part involves creating the database structure and filling it with data. The work must be kept in **two separate script files**, one for the schema and one for the data, so that it is organised and repeatable. Running the schema script followed by the population script on a fresh server must reproduce your database exactly.

3.1 Schema Creation

Your schema script must, when executed:

1. drop any existing copy of your database (or its contents) and create it afresh;
2. select or switch to your database;
3. define all structures, constraints and indexes of your model.

Relational (MySQL) – `schema.sql`

Run with `mysql -u <user> -p < schema.sql`. Pay meticulous attention to:

- **Data Types:** choose the most appropriate type for each attribute (e.g. `INT`, `VARCHAR(255)`, `TEXT`, `DATE`, `DECIMAL(10, 2)`, `ENUM`).

- **Primary Keys:** define a primary key for every table.
- **Foreign Keys:** correctly define every foreign key (`FOREIGN KEY ... REFERENCES ...`).
- **Referential Integrity:** specify `ON DELETE` and `ON UPDATE` actions (`CASCADE`, `SET NULL`, `RESTRICT`) as your world's rules require.
- **Constraints:** enforce integrity with `NOT NULL`, `UNIQUE` and `CHECK` where applicable.

Document (MongoDB) – `schema.js`

Run with `mongosh schema.js`. Pay attention to:

- **Collections and Validation:** create each collection with `db.createCollection()` and a `$jsonSchema` validator that specifies `bsonType`, `required` fields, `enum` values and `minimum`/`maximum` bounds where applicable.
- **Keys:** use `_id` sensibly, and create unique indexes (`createIndex(..., {unique: true})`) for natural keys.
- **Embedding vs. Referencing:** embed data that is always read together with its parent, and reference data that is shared or grows without bound. Justify your choices in the README.
- **Referential Integrity:** MongoDB does not enforce references. Decide what each delete does (cascade, nullify or forbid), implement it in your application, and document it.
- **Indexes:** index the fields your queries filter, sort or `$lookup` on.

Graph (Neo4j) – `schema.cypher`

Run with `cypher-shell -u <user> -p <password> -f schema.cypher`, or paste it into Neo4j Browser. Pay attention to:

- **Model:** represent entities as labelled nodes and relationships as typed, directed relationships. Place attributes on the node or on the relationship, as appropriate.
- **Keys:** create uniqueness constraints (`CREATE CONSTRAINT ... REQUIRE ... IS UNIQUE`) for every identifying property.
- **Constraints:** use property existence and type constraints where your Neo4j edition supports them. Otherwise, enforce these rules in your application and document where.
- **Data Types:** store values with appropriate Cypher types (`INTEGER`, `FLOAT`, `STRING`, `DATE`, `BOOLEAN`, lists).
- **Deletion Semantics:** for each node type, decide whether a delete should cascade (`DETACH DELETE`) or be blocked while relationships exist, and document it.
- **Indexes:** index properties that your queries look up or filter on.

3.2 Data Population

Your population script (`populate.sql`, `populate.js` or `populate.cypher`) must insert all the data your application needs.

- **Data Quality:** Insert data from the actual website your mini-world is modeled on. You can use GenAI tools to extract data. Avoid placeholder data (e.g. `'test1'`, `'test2'`), and generic data not present on the site.
- **Data Quantity:** insert enough data to demonstrate every functional requirement effectively. Include data that will and will not match your joins, `$lookup`s or graph patterns, and data that can safely be updated or deleted.
- **Insertion Order:**
  - MySQL: insert into parent tables before the child tables that reference them.
  - MongoDB: insert referenced documents before the documents that reference their `_id`s.
  - Neo4j: create nodes before the relationships that connect them, using `MATCH` to find the endpoints.

4 Part B: Web Application

4.1 Database Connectivity

Your backend must be written in **Python** and must connect to your database only through the Python driver for your paradigm (see the table in Section 2). For this project, you may place credentials in a configuration function, a `config.py` module or environment variables.

4.2 Web Interface

You must build a web application:

- **Backend:** Python, using any web framework (e.g. Flask, FastAPI or Django). If you use Django, you may not use its ORM (see Section 4.3).
- **Frontend:** any technology you like: server-rendered templates (e.g. Jinja), plain HTML/CSS/JavaScript, or a framework such as React, Vue or Svelte. A separate frontend must get all its data from your Python backend's API.
- **Server-side only:** the frontend talks to your backend, and only the backend talks to the database. The browser must never connect to the database directly.
- **Modelled on the original website:** Your interface should mostly resemble that site in its main pages, navigation and user flows. It does not need to match exactly or reproduce every feature.
- **Functional requirements in the UI:** All queries and updates should be reachable through a natural interaction on the site, such as a search bar, a filter, a detail page, a form submission or a delete button. A single page with a free-text box for typing raw queries does *not* meet this requirement.
- **Results and errors:** render results in the page as tables, cards or lists, and show errors (constraint violations, missing records, invalid input) as clear messages.
- **Hosting:** running locally (e.g. `http://localhost:5000`) is enough, deployment is not required.

Evaluation: The web interface is required, but grading focuses on the database implementation, the data quality, the correctness of the query-backed functional requirements, and how well the interface exposes them.

4.3 Functional Requirements Implementation

You must implement at least **five distinct query (read) operations** and **three distinct update (write) operations**. The write operations must include at least one create, one update and one delete:

Operation	MySQL	MongoDB	Neo4j
Create	`INSERT`	`insert_one` / `insert_many`	`CREATE` / `MERGE`
Update	`UPDATE`	`update_one` / `update_many`	`SET` / `REMOVE`
Delete	`DELETE`	`delete_one` / `delete_many`	`DELETE` / `DETACH DELETE`

Query Purity: You must write raw queries in your database's native query language:

- **SQL and Cypher:** queries are written as strings in your Python code and run through the driver.
- **MongoDB:** queries are written as explicit filter documents, update documents and aggregation pipelines (Python dicts and lists) passed to `find`, `aggregate`, `update_one` and similar methods. These count as raw MQL.
- **Not permitted:** ORMs, ODMs, OGMs and query builders, or any library that generates the query for you. Examples: SQLAlchemy (ORM and query builder), the Django ORM, Peewee, MongoEngine, Beanie, Neomodel, py2neo OGM and `pandas.to_sql`.
- **Do the work in the database:** filtering, joining, aggregation and traversal should be done in the query. Do not fetch full tables or collections and then filter them in Python.

Parameterised Queries: Do not build a query by inserting user input into a query string. Pass user input as parameters using your driver's mechanism:

- **MySQL:** `%s` placeholders in PyMySQL and `mysql-connector-python`.
- **Neo4j:** `$name` parameters in Cypher.
- **MongoDB:** fixed filter documents with type-checked values. Never pass user-supplied JSON straight into a filter, and never pass user input to `$where`.

Incorrect and correct examples for each paradigm are given in *Project Phase 4: Boilerplate Code*.

5 Submission Guidelines

Your final submission must be a single `.zip` file named `<team_number>.zip`. It must contain the following items at its root level.

Required File Structure:

```
<team_number>.zip
|-- <team_number>.mp4
|-- phase3.pdf
```

```
|-- README.md
`-- src/
    |-- schema.sql    | schema.js    | schema.cypher     (your paradigm's)
    |-- populate.sql | populate.js | populate.cypher   (your paradigm's)
    |-- app.py                  (Python backend entry point)
    |-- requirements.txt        (Python dependencies)
    |-- templates/, static/     (if you use server-rendered pages)
    |-- frontend/               (if you use a separate frontend)
    `-- (any other helper modules)
```

File-by-File Breakdown

1. **`<team_number>.mp4` (maximum 10 minutes):** a screen recording that demonstrates your project.
   - Show two windows at the same time, or switch between them clearly:
     - **Window 1:** your web application running in a browser.
     - **Window 2:** your database shell or GUI: the `mysql` client, `mongosh`, or `cypher-shell` / Neo4j Browser.
   - **Opening (about 30 seconds - 1 minute):** briefly tour the main pages and say which website your interface is modelled on.
   - **Demonstration flow for updates:** for each create, update and delete operation:
     (a) in the database shell, run a query showing the data before the operation;
     (b) in the web application, perform the operation through the UI;
     (c) return to the database shell and re-run the exact same query to show that the data changed correctly.
   - **Demonstration flow for queries:** run every query feature from the web interface and show that it returns the correct data.
2. **`phase3.pdf`:** the complete Phase 3 report on your Mini World.
3. **`README.md`:** this is a critical file. It must contain:
   - the paradigm(s) and DBMS you used, with versions;
   - your backend framework and frontend stack;
   - a **Design Changes** section that gives the team numbers of the teams that worked on your Mini World in Phases 2 and 3, and lists every significant change you made to the Phase 3 design (entities, tables, attributes, relationships or constraints added, removed or altered), each with a one-line reason;
   - setup and run instructions: how to load the schema and population scripts, install dependencies (`pip install -r requirements.txt`, plus any frontend install or build steps), start the application and open the site (the URL);
   - the **Schema Mapping** section, if you used MongoDB or Neo4j (see Section 2);
   - a clear numbered list of every feature your application offers. For each, state where it is in the UI (page or URL), how to trigger it, and what it does. For example: "1. Search worlds by name: Home page search bar (`/search?q=...`). Shows every world whose name contains the search term, with its description."

   The feature list must match the order of your video demonstration.

4. **`src/` directory:** all your source code.

   - your schema script: the final, executable schema creation script for your paradigm;
   - your population script: the final, executable data population script;
   - `app.py` (or a similarly named entry point) and `requirements.txt`;
   - all templates, static files, frontend source and helper modules your application depends on.

Good luck with Phase 4!